Université Tahri Mohamed de Béchar
Faculté des sciences exactes
Département des Mathématiques et Informatique
2ème Année Ingénieur Informatique

Matière:

Algorithmiques et complexité

Cours n° 05

Enseignant : Benyamina Ahmed

Matière: Algorithmiques et complexité

Programme de la matière :

I- Rappels

II- Notion de complexité algorithmique

III- Algorithmes de tri

IV- Les arbres

1. Introduction
2. Définition
3. Arbres binaire
4. Etude de complexité

V- Les graphes

Les arbres
1.Introduction

Les **arbres** sont des structures de données fondamentales en informatique, qui permettent de représenter et de manipuler des **données hiérarchiques**.

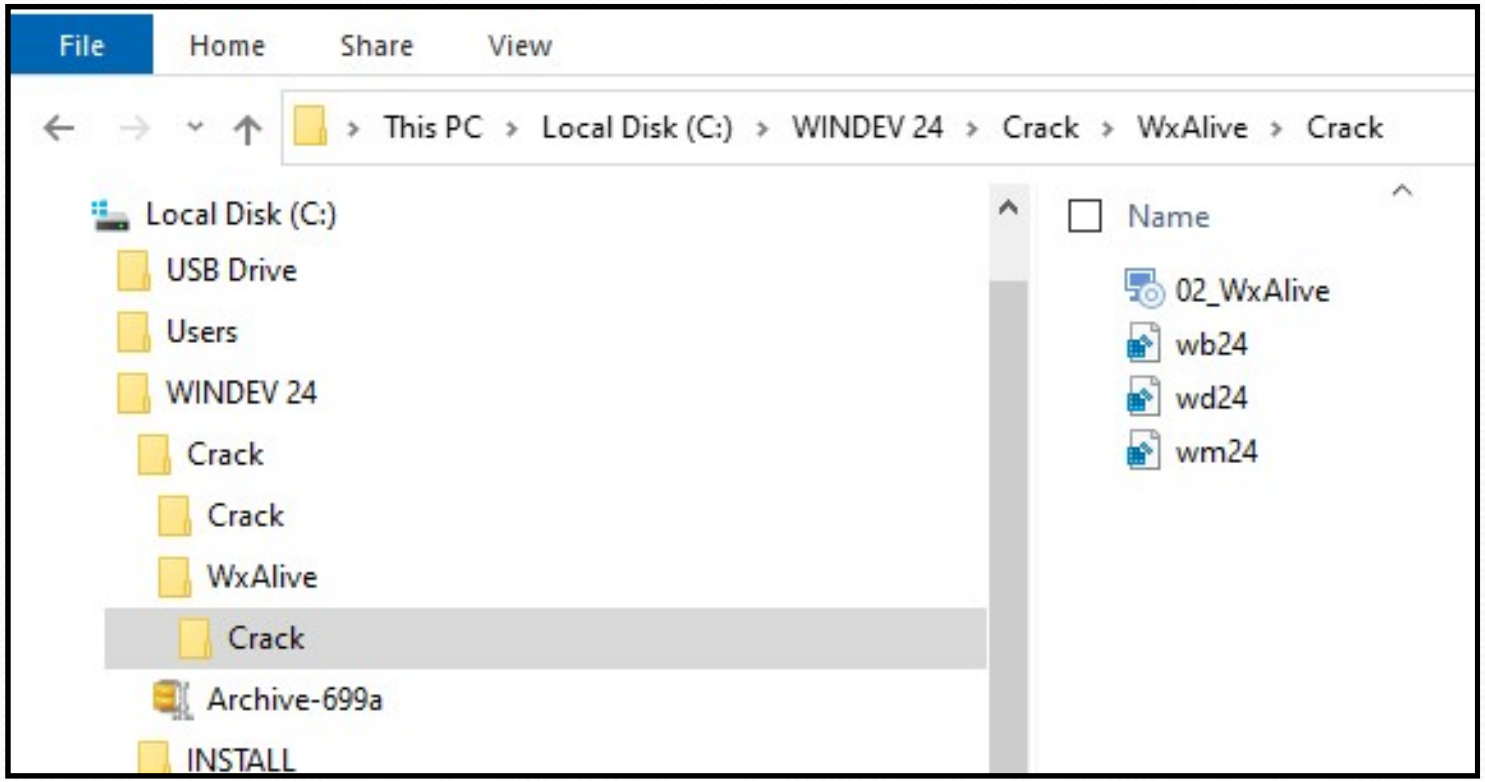
Exemples:

1. Organisation du système de fichiers d'un ordinateur est arborescente.
2. Le DOM (Document Object Model) du langage HTML
3. **Structures hiérarchisées :**
   - Organisations hiérarchiques,
   - Arbres familiales
   - Expressions algébriques,
   - Compilation (arbres syntaxiques)
   - Intelligence artificielle : arbres de décision,
   - Organisation des cinq continents de la terre.
   - Découpage d'un livre en chapitres, sections, etc

Les arbres
1.Introduction

Exemple 1:

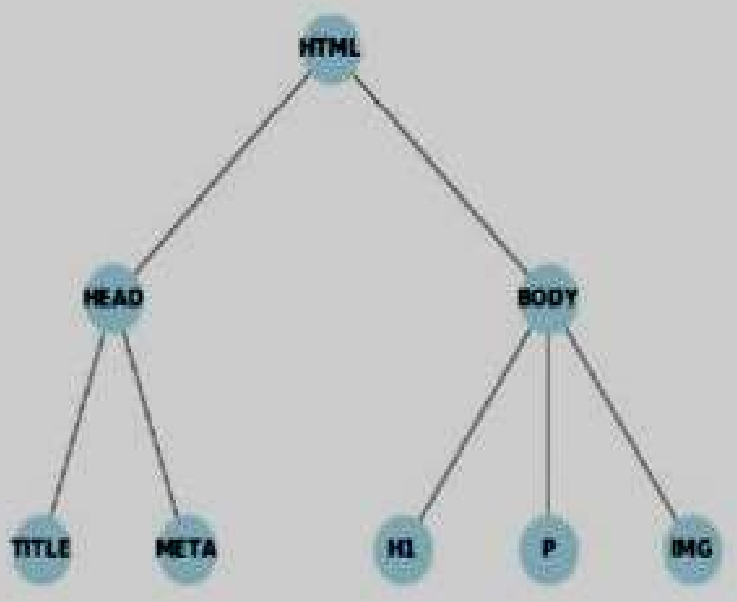
- Organisation du système de fichiers d'un ordinateur est arborescente. Les fichiers sont organisés de manière hiérarchique à partir d'une racine, à laquelle peuvent être rattachés des répertoires (dossiers) et des fichiers, les répertoire pouvant eux-mêmes de manière récursive contenir d'autres répertoire et/ou fichiers.



Les arbres
1.Introduction

Exemple 2:
- Le DOM (Document Object Model) modèle objet de document du langage HTML représente un document web de manière structurée sous forme d'un arbre hiérarchique

```
<!DOCTYPE html>
<html lang=fr dir="ltr">
  <head>
    <meta charset="utf-8">
    <title>Bienvenue</title>
  </head>
  <body>
    <h1>Bonjour</h1>
    <p>Bienvenue sur notre site internet</p>
    <img src="logo.png" alt="notre_logo">
  </body>
</html>
```



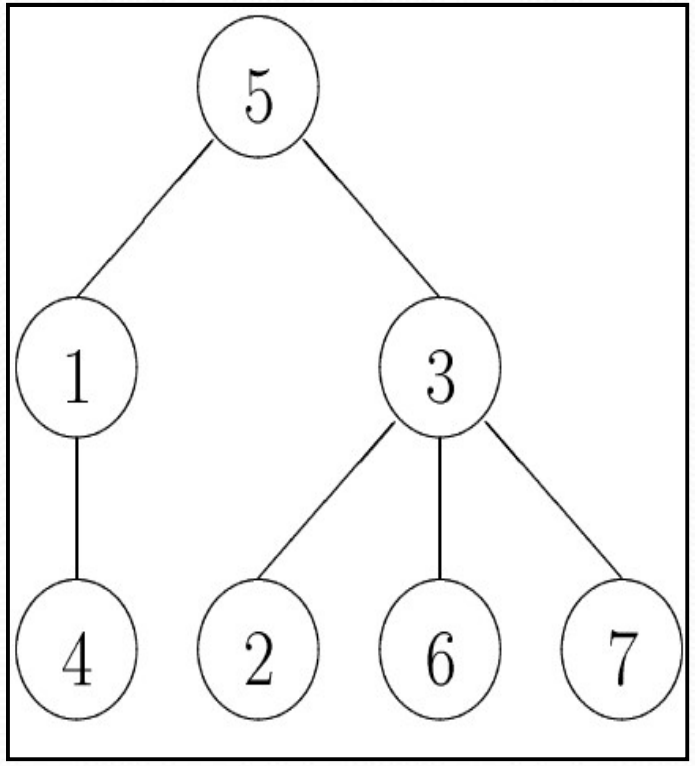
Les arbres

2.Définitions

Arbre: Un arbre est une structure de données composés de **nœuds** reliés entre eux par des **branches** : chaque nœud pointe vers un ensemble de nœuds qui sont ses fils (les enfants), et tous nœuds sauf un ont un père et un seul; le nœud sans père s'appelle le nœud **racine**.

Exemple: La figure ci-après montre graphiquement la présentation d'un d'arbre :

- La **racine** est toujours représentée en haut et les branches dirigées vers le bas.
- Sur cet arbre on a **sept nœuds**,
- Le nœud 5 est la **racine** et a deux **fils** : 1 et 3,
- Le nœud 1 a un fils : 4,
- Le nœud 3 trois fils : 2, 6 et 7,
- Le père de 2 est 3, celui de 3 est 5,
- Un nœud sans fils s'appelle une **feuille** (4, 2, 6 et 7)
- Un nœud qui n'est pas une feuille est un nœud interne : ici 1, 3 et 5.



Les arbres

2. Définitions

- Ancêtre d'un nœud :

Les ancêtres de 2 sont 3 et 5.

- Descendant d'un nœud :

Le nœud 5 admet les six nœuds comme descendant

-Profondeur d'un nœud :

C'est la longueur du chemin qui le lie à la racine

Le nœud 5 est donc de profondeur « 0 »,

Les nœuds 1 et 3 de profondeur « 1 »

Les nœuds 4, 2, 6 et 7 de profondeur « 2 ».

-La hauteur d'un arbre:

Est la profondeur maximale de ses feuilles

La hauteur de cet arbre est « **2** »

-Le degré d'un nœud:

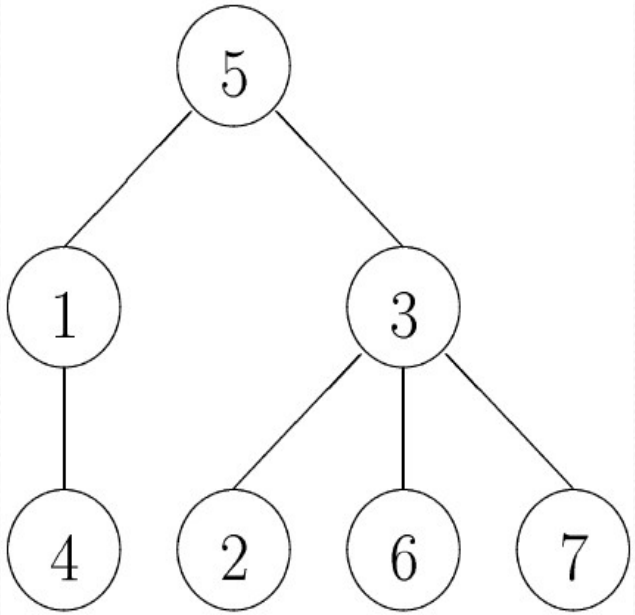
Est égal au nombre de ses fils

Le nœud 5 est de degré « 2 ».

Le nœud 1 de degré « 1 ».

Le nœud 3 de degré « 3 ».

Les feuilles (4,2,6,7) de degré « 0 ».



Un arbre atomique :

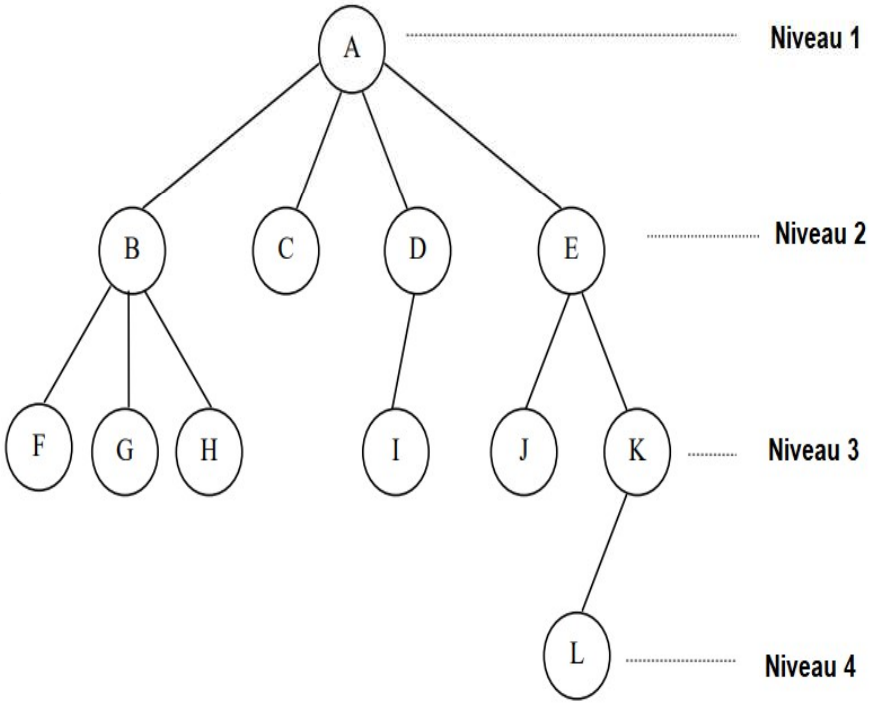
Un arbre est dit atomique s'il contient un nœud « ***feuille*** »

Cet arbre de l'unique nœud 5 est dit ***atomique***

Représentation d'un arbre :

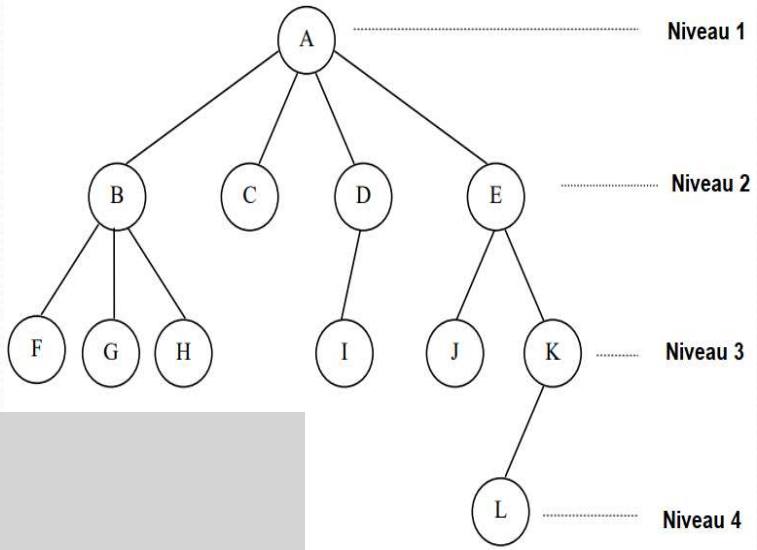
La représentation courante d'un arbre, adaptée à son stockage dans la mémoire d'un ordinateur, est une liste.

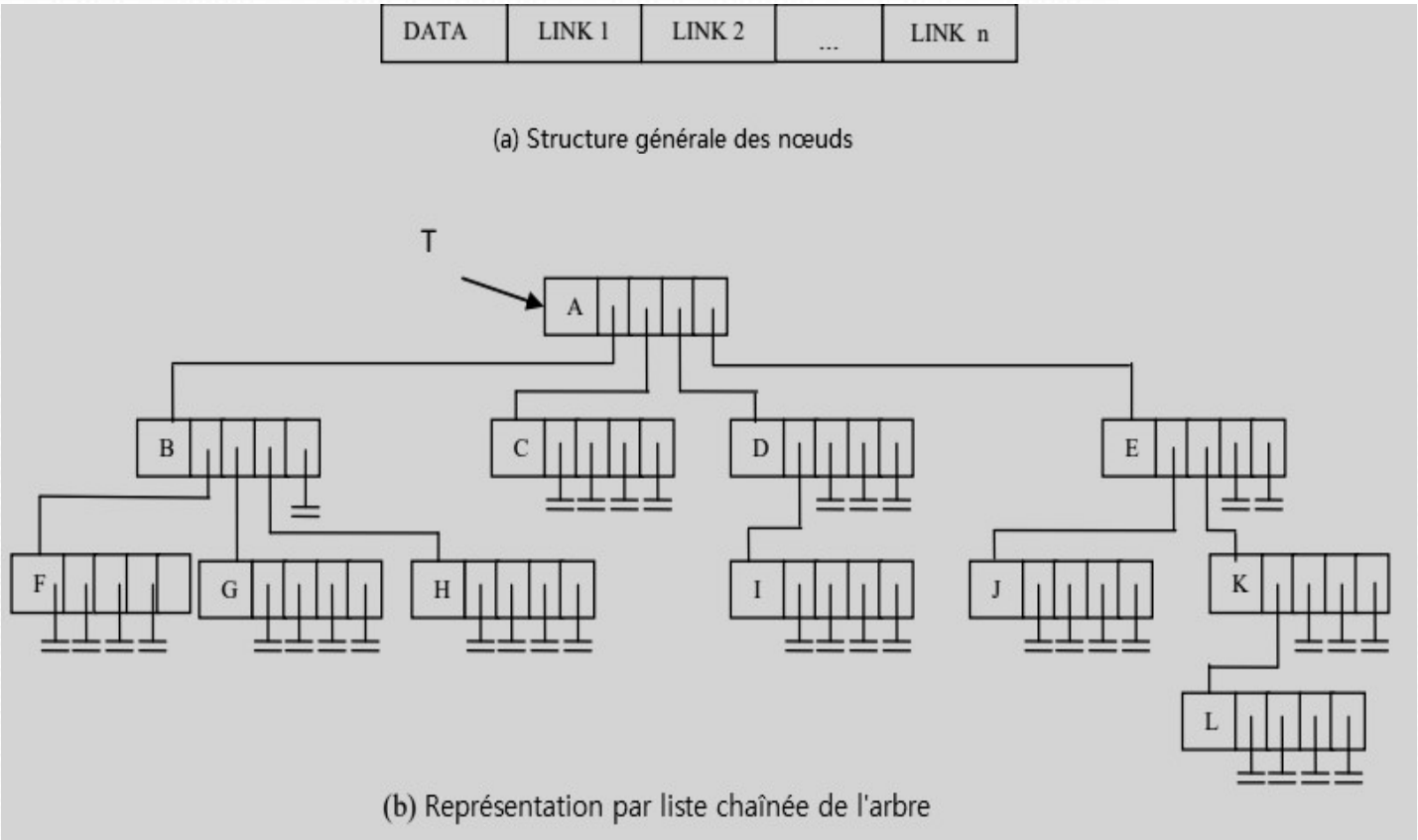
Exemple : L'arbre ci-après pourrait être représenté sous sa forme de liste par (A (B(F,G,H), C, D(I), E(J,K(L)))). Le nœud racine vient en premier suivi de la liste des sous-arbres du nœud. Ceci est répété pour chaque sous-arbre de l'arborescence. Cette liste, sous forme d'arbre, ouvre la voie à un représentation naïve de l'arbre sous forme de liste chaînée.



Représentation d'un arbre :

La structure générale des nœuds d'un arbre est illustré à la figure ci-après:





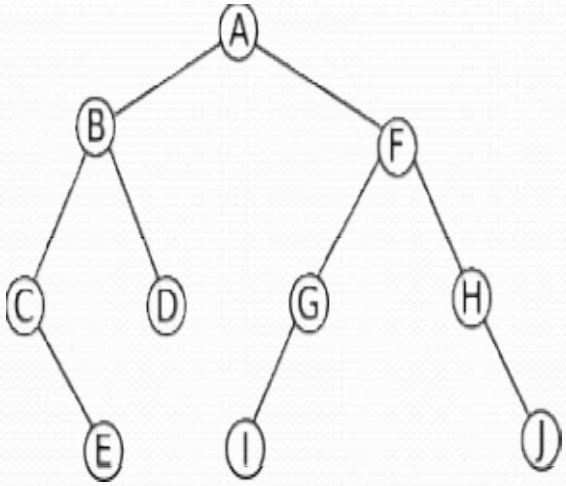
2. Définition: Les types d'arbres

Il existe de nombreux types d'arbres dans le domaine des structures de données. Voici quelques-uns des types d'arbres les plus courants :

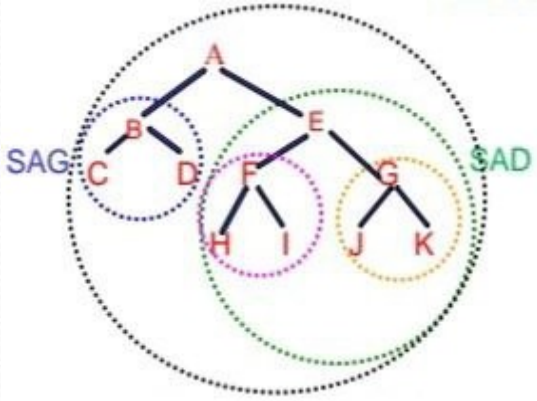
1. Arbres binaires :
2. Arbres de recherche binaire
3. Arbres généalogiques
4. Arbres d'expression :

2. Définition: Les types d'arbres

1- Arbre binaire: Un arbre binaire est une structure de données hiérarchique composée de nœuds, où chaque nœud a au plus deux enfants, généralement appelés "gauche" et "droite". Chaque nœud d'un arbre binaire peut avoir un maximum de deux sous-arbres, un à gauche et un à droite. La structure d'un arbre binaire est telle que chaque nœud a un seul parent, à l'exception du nœud racine qui n'a pas de parent.

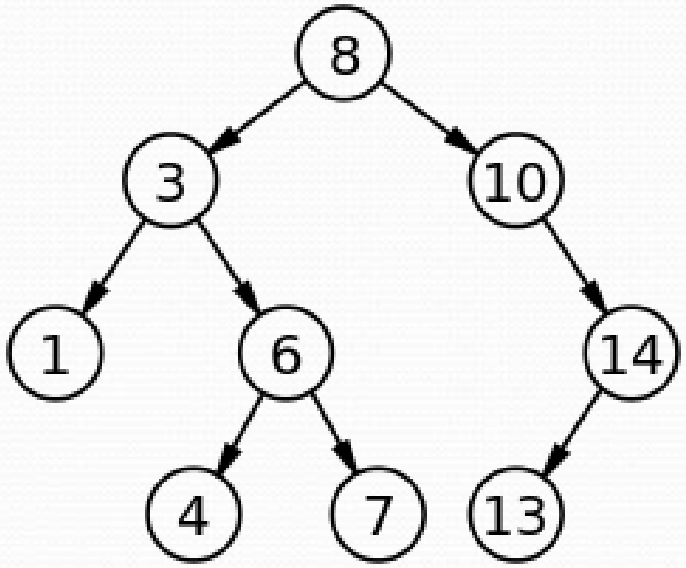


Les arbres binaires ont une définition récursive naturelle



2. Définition: Les types d'arbres

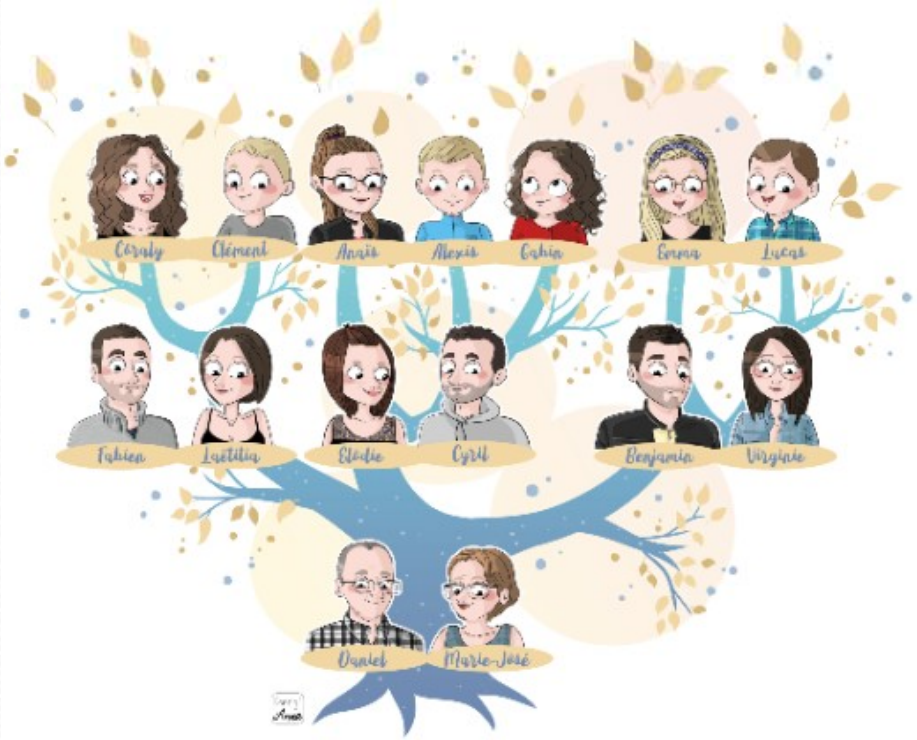
2- Arbre binaire de recherche: Un type particulier d'arbre binaire dans lequel les nœuds sont organisés de manière à faciliter la recherche. Pour chaque nœud, tous les nœuds du sous-arbre gauche sont inférieurs, et tous les nœuds du sous-arbre droit sont supérieurs. Ces propriétés permettent une recherche, une insertion et une suppression très efficaces. En fait, le temps de recherche moyen pour un arbre de recherche binaire est directement proportionnel à sa hauteur : **O(h)**



Les arbres

2. Définition: Les types d'arbres

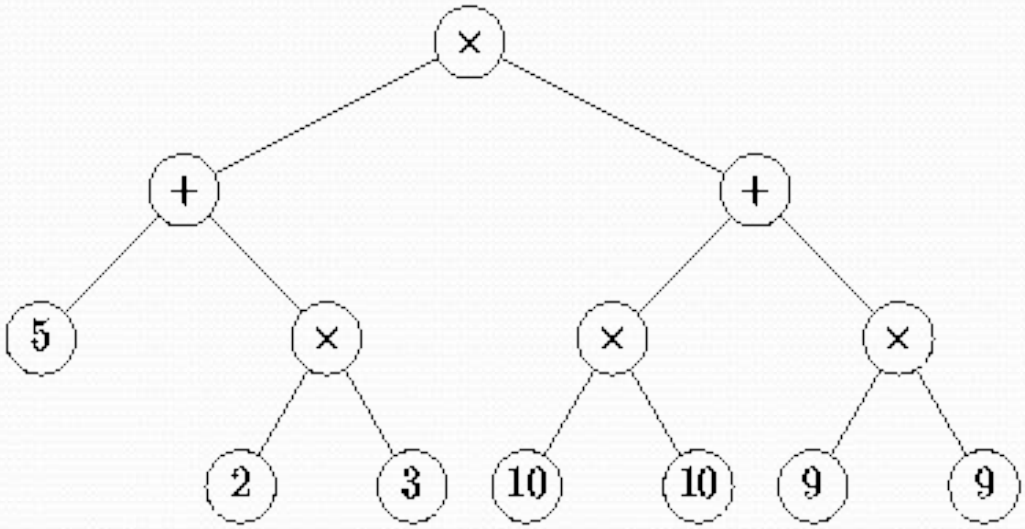
3. Arbres généalogiques : Les arbres généalogiques sont utilisés pour représenter les relations familiales, avec les nœuds représentant des individus et les liens entre eux indiquant des relations parent-enfant.



2. Définition: Les types d'arbres

4. Arbres d'expression : Les arbres d'expression sont utilisés pour représenter des expressions **mathématiques** ou **arithmétiques**.
L'arbre est hétérogène, le genre d'information est pas le même sur chaque nœud (opérateur, opérande).
Les feuilles contiennent les opérandes,
Les nœuds internes contiennent les opérateurs.
Ils permettent l'évaluation et la simplification des expressions.

Exemple:
Voici l'expression suivante :
(2*3+5)*(10*10+9*9)



A- Parcours d'un arbre binaire :

Il existe trois principaux types de parcours :

1. Le parcours ***préfixe*** ou préordre: Preorder Traversal
2. Le parcours ***infixe*** ou en ordre: Inorder Traversal
3. Le parcours ***postfixe*** ou postordre : Postorder Traversal).

A-1- Parcours Préfixe (Preorder Traversal) :

Le parcours de préfixé est un algorithme permettant de visiter (traiter) tous les nœuds d'un arbre binaire. L'algorithme visitera d'abord ce nœud (la « racine » de son sous-arbre), puis il visitera de manière récursive tous les nœuds de son sous-arbre de gauche, et enfin il visitera de manière récursive tous les nœuds de son sous-arbre de droite.

- Visite le nœud courant ⇒ **R**
- Visite le sous-arbre gauche ⇒ **G**
- Visite le sous-arbre droit ⇒ **D**

Utilisation: La création d'une copie de l'arborescence,

A-1- Parcours Préfixe (Preorder Traversal) : (RGD)

3. Les arbres binaires : parcours

A-1- Parcours Préfixe (Preorder Traversal) : (RGD)

Algorithme : L' approche de parcours présenté est récursive utilisant une pile d'appels système pour garder une trace des nœuds à visiter ensuite. C'est une méthode simple et directe, mais elle peut entraîner des erreurs de débordement de pile si l'arborescence est très profonde.

```
def recursive_pre_order_traverse (node):
    if node is None:
        return
    node. visit ()
    recursive_pre_order_traverse (node . left)
    recursive_pre_order_traverse (node. right)
```

Oú

```
class Node:
    def _init__ (self, key) :
        self.key = key
        self.left = None
        self. right = None
    def visit (self):
        print (self . key , end=" ")
```

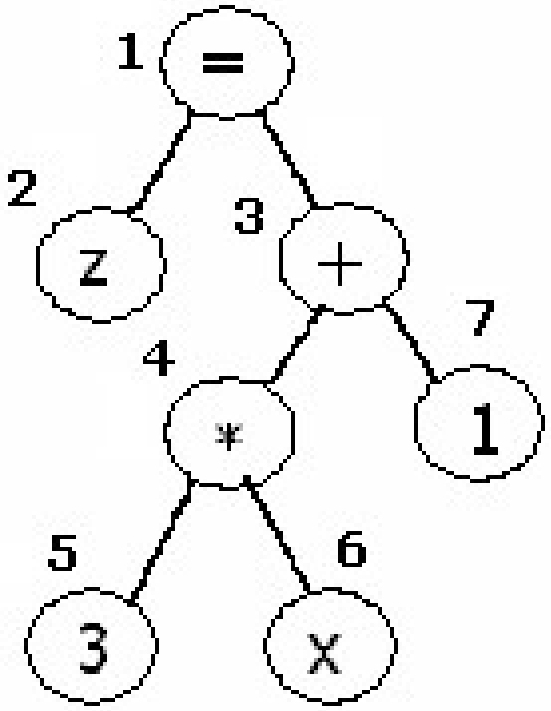
A-1- Parcours Préfixe (Preorder Traversal) : (RGD)

Complexité temporelle : O(n), où n est le nombre de nœuds dans l'arborescence. En effet, chaque nœud de l'arborescence est visité exactement une fois pendant le parcours.

A-1- Parcours Préfixe (Preorder Traversal) : (RGD)

<u>Exemple</u> 1: Z=3x+1

Représentation schématique :



Représentation préfixée :=z+*3 X 1

A- Parcours d'un arbre binaire :

Il existe trois principaux types de parcours :

1. Le parcours ***préfixe*** ou préordre: Preorder Traversal
2. **Le parcours *infixe*** ou en ordre: Inorder Traversal
3. Le parcours ***postfixe*** ou postordre : Postorder Traversal).

A-2- Parcours Infixe (Inorder Traversal):

- Visite le sous-arbre gauche ⇒ **G**
- Visite le nœud courant ⇒ **R**
- Visite le sous-arbre droit ⇒ **D**

Utilisation:

1. **Affichage trié :** produit une séquence triée des éléments.
2. **Validation de l'expression arithmétique :** récupérer l'expression dans l'ordre correct.

A-2- Parcours Infixe (Inorder Traversal): (GRD)

A-2-Parcours Infixe (Inorder Traversal): (GRD)

Algorithme :

L' approche de parcours infixe présenté est récursive.

```
def recursive_in_order_traverse (node):
    if node is None:
        return
    recursive_in_order_traverse (node. left)
    node.visit()
    recursive_in_order_traverse (node . Right)
```

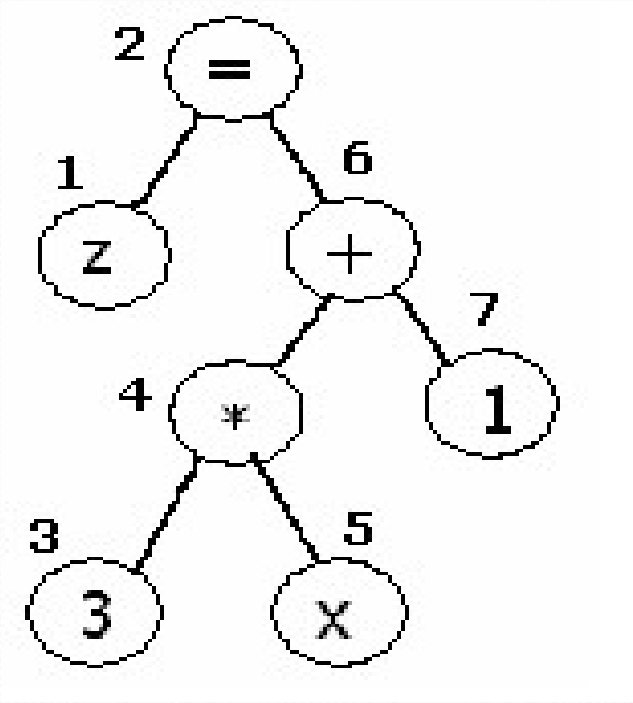
A-2-Parcours Infixe (Inorder Traversal):(GRD)

Complexité temporelle : O(n), où n est le nombre de nœuds dans l'arborescence.

A-2-Parcours Infixe (Inorder Traversal): (GRD)

Exemple: Z=3x+1

Représentation schématique :



Représentation infixée : Z=3*X+1

A- Parcours d'un arbre binaire :

Il existe trois principaux types de parcours :

1. Le parcours ***préfixe*** ou préordre: Preorder Traversal
2. Le parcours ***infixe*** ou en ordre: Inorder Traversal
3. **Le parcours *postfixe*** ou postordre : Postorder Traversal).

A-3- Parcours Postfixe (Postorder Traversal):

- Visitez le sous-arbre gauche ⇒ **G**
- Visitez le sous-arbre droit ⇒ **D**
- Traitez le nœud courant ⇒ **R**

<u>Utilisation</u>:

Utilisé pour l'évaluation d'une expression arithmétique

Illustrons cela avec un exemple d'expression postfixe : **23*5+**.

1.Parcours de gauche à droite :

- 2 (opérande) -> empile 2
- 3 (opérande) -> empile 3
  - (opérateur) -> dépile 3 et 2, calcule 3 * 2, empile 6
- 5 (opérande) -> empile 5
  - (opérateur) -> dépile 5 et 6, calcule 5 + 6, empile 11

A-3- Parcours Postfixe (Postorder Traversal): (GDR)

A-3- Parcours Postfixe (Postorder Traversal): (GDR)

Algorithme :

L' approche de parcours infixe présenté est récursive.

```
def recursive_post_order_traverse (root) :
    if root is not None:
        recursive_post_order_traverse (root. left)
        recursive_post_order_traverse (root.right)
        root.visit ()
```

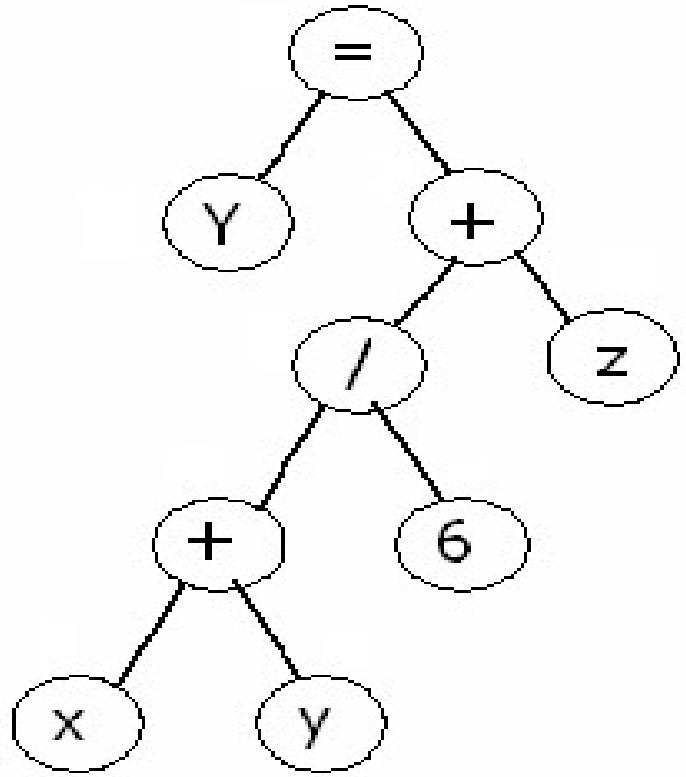
A-3- Parcours Postfixe (Postorder Traversal): (GDR)

Complexité temporelle : O(n), où n est le nombre de nœuds dans l'arborescence.

A-3- Parcours Postfixe (Postorder Traversal): (GDR)

Exemple :

1 - (y)(+)=
2 - y(/)(z)+=
3 - y(+)(6)/z+=
4 - y(x)(y)+6/+=



Représentation post fixée : **yxy+6/z+=**

Question: Evaluer l'expression arithmétique de cette notation postfixe?

Différence entre les structures de données: tableau et arbre

Caractéristiques	Tableau	Arbre
1. **Organisation des données**	Linéaire, stockage contigu en mémoire	Hiérarchique, nœuds liés sous forme d'arbre
2. **L'accès aux éléments se fait par des Relation entre les éléments**	Indices	Parcours spécifiques (inordre, préordre, postordre).
3. **Relation entre les éléments**	Éléments indépendants	Nœuds liés, relation parent-enfant
4. **Insertion/Suppression**	Décalage potentiel des éléments	Insertion/Suppression souvent éfficaces
5. **Recherche**	Accès direct par indice (O(1))	Recherche dépend du type d'arbre
6. **Utilisation**	Convient pour des collections simples	Utile pour représenter des relations hiérarchiques, structures de recherche, etc.